Lesson notes · for you

Heuristics and trade-offs

Optimization · Level 4 · Lesson 4

Time
25 min

Materials
Your phone, or a computer with Colab; A local map with distances between five to ten places

Learning goals

- Explain NP-hardness in plain words and why heuristics are used.
- Implement 2-opt and compare metaheuristics for escaping local optima.
- Measure the optimality gap and put fairness limits in the open.

Activity: Route race

1. Build (6 min): Mark 7 places and a start on paper with distances. Build a route by always going to the nearest unvisited place, and add up its length.
2. Improve (8 min): Wherever two legs cross, reverse the stretch between them (a 2-opt move) and recompute. Stop when no move shortens the route.
3. Check (6 min): Run the lesson's code or a brute-force search for the true best route, and compute your gap: (yours - best) ÷ yours.
4. Fair (5 min): Choose a limit on the longest ride from the start to any place. Does your route keep it? What would keeping it cost in total distance?

Use a real local map if you can; travel times work as well as distances. Work in pairs and swap routes to check each other's sums.

Discussion

- Why do companies accept routes nobody has proved optimal?
- Which metaheuristic would you try first for a timetable, and why?
- Who should decide the longest acceptable ride on a school bus?

Check yourself

- Your route is 52 km and a bound says 50 km. Write the gap, and one sentence a manager would understand.

Try it this week
Write 2-opt for a five-stop route in Python and check it against every possible route. Then add a limit on the longest ride for any passenger, solve again, and write one sentence on what the limit costs and who it protects.

Worksheet

Heuristics and trade-offs

Optimization · Level 4 · Lesson 4

Name _____ Class _____ Date _____

1 **You model the morning bus for five villages. Plan A is 2 km shorter in total, but one village's children ride 70 minutes; in plan B, 2 km longer, no child rides more than 40 minutes. Which should the model pick?**

- ◯ It depends on the agreed goals: show both and decide with those affected
- ◯ A: the shortest is the goal
- ◯ B, always

2 **Complete the program so it prints the optimality gap, in percent, of a 105 km route against a lower bound of 100 km.**

```
found = 105   # km, the route the heuristic found
bound = 100   # km, no route can be shorter
gap = (found - _____) / _____
print(round(_____ * gap, 1), "%")
```

Word bank: `0` `10` `found` `bound` `100`

3 **2-opt always finds the shortest route if you let it run until no move helps.**

◯ True ◯ False

4 **Five stops, distances in d, start route [0, 2, 1, 4, 3] (length 25). Fill in the 2-opt move: for each i < j, reverse the stretch r[i..j], and keep the new route if it is shorter.**

```
d = [[0, 3, 7, 6, 4], [3, 0, 4, 7, 6], [7, 4, 0, 3, 5],
     [6, 7, 3, 0, 2], [4, 6, 5, 2, 0]]
def length(r):
    return sum(d[r[i - 1]][r[i]] for i in range(len(r)))
r = [0, 2, 1, 4, 3]
for rounds in range(5):
    for i in range(1, 4):
        for j in range(i + 1, 5):
            pass  # reverse r[i..j]; keep it if shorter
print(r, length(r))
```

Write your program:

5 **Match each metaheuristic to its idea.**

Simulated annealing	• • • •	Forbids undoing recent moves for a while
Tabu search	• • • •	Runs local search from many random starts
Genetic algorithm	• • • •	Accepts some worse moves, less often as it cools
Multi-start	• • • •	Combines parts of good plans

✎ **Where would you accept a good-enough answer with a measured gap, and where would you insist on a proof of the best? What decides it?**

Check your answers

Heuristics and trade-offs

Optimization · Level 4 · Lesson 4

1 **It depends on the agreed goals: show both and decide with those affected**

Neither plan beats the other on both goals, so the choice is a judgement about what matters: 2 km of fuel and driving time against 30 minutes for one village's children.

2

```
found = 105   # km, the route the heuristic found
bound = 100   # km, no route can be shorter
gap = (found - bound) / found
print(round(100 * gap, 1), "%")
```

The gap is (found − bound) ÷ found = 5 ÷ 105, printed as 4.8 %.

3 **False**

It stops at a local optimum: no single 2-opt move helps, but a better route may need a different kind of move.

4

```
d = [[0, 3, 7, 6, 4], [3, 0, 4, 7, 6], [7, 4, 0, 3, 5],
     [6, 7, 3, 0, 2], [4, 6, 5, 2, 0]]
def length(r):
    return sum(d[r[i - 1]][r[i]] for i in range(len(r)))
r = [0, 2, 1, 4, 3]
for rounds in range(5):
    for i in range(1, 4):
        for j in range(i + 1, 5):
            new = r[:i] + r[i:j + 1][::-1] + r[j + 1:]
            if length(new) < length(r):
                r = new
print(r, length(r))
```

Reversing r[i..j] swaps two edges for two others.

5

Simulated annealing — Accepts some worse moves, less often as it cools

Tabu search — Forbids undoing recent moves for a while

Genetic algorithm — Combines parts of good plans

Multi-start — Runs local search from many random starts

All four escape local optima in different ways: by accepting bad moves, by memory, by recombination, or by trying again elsewhere.